FUTURESTACK HACKATHON | ROUND 2 OF 3

BrandLens

Pre-publication brand governance with rules first, cited AI second, and humans in control.

SUBMISSION READINESS

The code passes unit tests, lint, TypeScript, and a production webpack build. The hosted product is **not yet verified**: the configured Neon endpoint is a sample/unreachable host, so migrations, seed data, demo login, and the live review journey still need database-backed verification.

PROJECT LINKS

Working project: Pending a verified Vercel deployment. **Repository:** github.com/Utsav-83/BrandLens

Demo workspace: Aurora Pay. One-click creator, manager, reviewer, and admin accounts are available when the seeded database and `DEMO_MODE=true` are configured.

JUDGING READINESS SNAPSHOT

Criterion	Current evidence	Status
Core workflow (30)	Submission, analysis, finding decisions, review, approval, audit routes and smoke-test script are implemented; no successful live DB walkthrough yet.	Code ready; hosted proof pending
Technical depth (25)	Rules + citation-validated LLM, BM25, Prisma/Postgres, server guards, DB rate limits and hash-chain audit. Production webpack build passes.	Strong; DB verification pending
Round 1 progress (20)	Documented architecture and retrieval pivots, expanded workflow, source validation and audit integrity.	Documented
Round 3 plan (15)	Three substantial product improvements and measurable outcomes are listed on page 4.	Ready
Usability (10)	Seeded demo, role logins, sample draft, visible AI fallback, labels and keyboard-friendly controls are implemented; hosted usability still needs a pass.	Verify after deployment

Completed features

BrandLens checks drafts against organization-approved sources and routes findings through a human-controlled review workflow.

Ingestion: Text, DOCX, PDF, TXT and Markdown; magic-byte checks and bounded uploads. Digital-text PDFs work; scanned image-only PDFs need OCR.

Rules: Glossary replacements, unsupported absolutes, numeric fact checks, tone/slang, readability and accessibility-related text checks run without an LLM.

Retrieval and AI: BM25 selects source passages. Provider-neutral LLM calls use environment configuration; citation quotes and proposed numbers are checked against retrieved sources. Failures produce visible rules-only results.

Human workflow: Creators accept, edit or reject findings and submit for review. Managers/reviewers request changes or approve. Server-side roles, ownership boundaries and separation of duties protect actions.

Audit and operations: PostgreSQL via Prisma 7, DB-backed rate limits, SHA-256 audit chain with Postgres transaction locks, integrity verification and bounded-batch audit reads. Audit views are manager/reviewer/admin only and omit payload text.

Judge onboarding: Seeded Aurora Pay sources, four demo roles, sample flawed copy, and explicit loading, empty, error and AI-unavailable states.

TECHNOLOGY AND IMPLEMENTATION

Layer	Implementation
Web	Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4; Node.js route handlers and server components.
Data	PostgreSQL/Neon target, Prisma 7 custom prisma-client output, @prisma/adapter-pg runtime; DIRECT_URL for migrations.
Identity	jose HS256 JWT in httpOnly sameSite=lax cookie, bcryptjs, server-side roles and ownership checks.
Review	Deterministic TypeScript rules, BM25 retrieval, provider-neutral fetch LLM, 20-second timeout, JSON parsing, source quote validation and rules-only fallback.
Documents	mammoth for DOCX, unpdf for PDF text streams, file signature checks and size limits; image-only OCR is not implemented.
Quality	Vitest: 292 passed, 1 skipped. ESLint and TypeScript no-emit pass. Production webpack build passes. Postgres concurrency test requires a separate test database.

Security, reliability, and release status

Security controls

Authorization: Route handlers and server-rendered pages enforce identity and role checks; proxy.ts is only a convenience redirect. Creators can edit their own drafts; admins can edit any draft in their workspace. Managers and reviewers see only review-related submissions.

Review boundary: A creator cannot approve their own submission, including when the creator also has an admin role. Audit pages and APIs are limited to managers, reviewers and admins; event payloads are omitted from audit views.

Request safety: Zod validates API inputs. Mutating requests check Origin. JWTs use httpOnly, sameSite=lax cookies, secure in production. Production HSTS is configured; development-only unsafe-eval is excluded from production CSP.

Data handling: Draft text and secrets are not intentionally logged. The analysis route has a 60-second maximum duration. PostgreSQL rate limits work across serverless instances; demo login allows 30 attempts per IP per 10 minutes for shared event networks. The limiter fails open when PostgreSQL is unavailable, favoring demo availability over abuse protection during an outage.

VERIFICATION RESULTS

Check	Result
Automated suite	292 passed, 1 skipped; the skipped concurrency integration test needs a dedicated test database.
Lint and types	ESLint and TypeScript no-emit passed.
Production build	The package build script now selects webpack because Turbopack's CSS Node worker exited before connecting on this Windows workstation, including outside the sandbox. The exact npm run build command passes with webpack; Vercel's Linux build result remains unverified.
Database	Prisma migrate status failed against the configured sample Neon endpoint. No valid production or separate test database has been verified.
Deployment	The current implementation and submission updates are pushed to main. Vercel deployment and public smoke test are still unverified.

REMAINING RELEASE GATES

1. Set valid pooled `DATABASE_URL` and direct `DIRECT_URL` for Production in Vercel; keep secrets out of Git and chat. 2. Set a separate `TEST_DATABASE_URL` for integration tests. 3. Apply migrations and seed the production demo workspace. 4. Redeploy and confirm the current Vercel build is ready. 5. Run the remote smoke test and two-user ownership check, then update the project link on page 1.

Never report the project as live until the public URL and core creator-to-reviewer workflow pass.

Plan changes and Round 3 improvements

Changes from the original plan

FastAPI + React to unified Next.js: Reduced cross-origin session and deployment complexity; routes, validation, server rendering and shared types now live in one TypeScript app.

pgvector to BM25: Prioritized deterministic exact-term retrieval for product names and governance terms, avoiding embedding latency in the core workflow.

Hard LLM dependency to rules-first fallback: Brand checks continue if the provider is absent, fails or times out; the UI explains that AI review is unavailable.

Basic audit rows to hash chain: Entries link cryptographically and use a PostgreSQL advisory transaction lock for workspace writes.

Visual/OCR review deferred: Round 2 focused on text governance and reliable document parsing; scanned/image-only documents are not analyzed.

THREE MAJOR ROUND 3 IMPROVEMENTS

Improvement	Product addition and success measure
1. Visual creative and accessibility governance	Accept image/design exports, extract text with OCR, inspect logo/color use and contrast, and anchor findings to image regions. Measure OCR and contrast accuracy against labelled creative assets.
2. Governance intelligence across sources	Detect contradictory or superseded facts, alert on expiring claims, and surface recurring policy errors. Add source lineage and manager resolution actions; measure contradiction detection on a labelled source set.
3. Private inference and measurable AI quality	Add local open-model inference through Ollama/vLLM and a versioned golden evaluation suite with precision, recall and citation-grounding metrics. Verify private mode makes zero outbound LLM calls.

Judge quick-start after release gates pass

1. Continue as Creator on /login. 2. Start a draft and choose Load Sample Flawed Content. 3. Save and analyze; inspect source-backed findings and any rules-only notice. 4. Accept/edit/reject fixes and submit for review. 5. Sign in as Reviewer, inspect the queue, request changes or approve, then open /audit and verify chain integrity.